Overwatch

AI Talent Intelligence & Recruitment Platform

Team: The Big Oh's · Idea Submission

Table of Contents

1. Executive Summary & Core Value Proposition

1.1 Executive Summary

Hiring runs on documents nobody can verify. A resume asserts five years of Python. A certificate asserts a credential. Recruiters read all of it and guess. The candidates who get through are often the ones who write the best claims, not the ones who wrote the best code.

Overwatch inverts that. It scores people on artifacts that are expensive to fake: commit history, merged pull requests to repositories they don't own, code written under a timer, answers given in a live interview. The resume becomes the least trusted input in the pipeline rather than the primary one.

Four things work together. A **Talent Score** engine reads GitHub, resumes, certificates and assessment results, then produces nine sub-scores plus a weighted composite, each carrying the evidence that produced it. A **job matching engine** scores candidate–job pairs on skill overlap, semantic similarity, experience fit and job-specific talent alignment. A **hackathon-to-hiring pipeline** analyzes team submissions and pushes top performers into recruiter feeds. A **trust layer** runs certificate verification, code plagiarism detection and duplicate-profile matching.

One rule shapes the whole system: **nothing silently moves a number.** A fraud flag has zero effect on a candidate's score until a human reviews and upholds it. When a signal is missing, its weight is redistributed across the signals that exist and the candidate is told which ones were dropped. Nothing is quietly zero-filled, and no score is shown as a bare number without the components behind it.

1.2 What Overwatch Is, In Plain Terms

Overwatch looks at what a person has actually built, and turns that into a score a recruiter can trust and a candidate can see the reasoning behind.

Who	What they do today	What they do with Overwatch
Candidate	Writes a resume and hopes someone believes it	Connects GitHub, takes a timed assessment, gets a score with the evidence attached, and sees exactly which skills are missing for the role they want
Recruiter	Reads hundreds of resumes, keyword-searches, guesses	Gets a ranked shortlist per job where every number opens into the reasoning behind it
Hackathon organizer	Runs an event, picks winners, event ends	Runs the event on the platform; strong performers automatically reach recruiters watching
Judge	Scores submissions on a spreadsheet	Gets a queue with the rubric built in and pitch analysis already prepared
Admin	No central oversight of the process	Reviews fraud flags, manages roles, audits every action taken

A worked example. A final-year student connects her GitHub. The system reads her repositories, notices she has written substantial Rust and TypeScript, and cross-checks that against the two certificates she uploaded. Her resume claims "expert in machine learning" but no repository supports it, so that claim is flagged as unverified to her, not silently accepted or silently dropped. She takes a timed coding assessment in the browser. Her Talent Score comes out at 74, with a note that three of nine signals are missing because she has no hackathon or open-source history yet.

A recruiter hiring for a Rust backend role searches in plain language. She ranks high, not because of her resume wording, but because her *verified* Rust work matches the role. The recruiter opens her score and sees which repositories produced it. Six months later, that reasoning is still on record.

1.3 The Problem

For recruiters. Keyword matching against resume text is the industry default. It rewards keyword stuffing, misses candidates who describe the same skill differently, and produces a ranked list with no explanation attached. A recruiter challenged on a decision six months later has nothing to reconstruct it from.

For candidates. A strong engineer with a thin resume loses to a weaker one with a better-written resume. New graduates and career-switchers have no assessments and little commit history, so systems that treat missing data as zero score them near the bottom regardless of ability.

For hackathon organizers. Events generate exactly the signal recruiters want: real code, built under time pressure, judged by experts. Then the event ends and all of it is thrown away. There is no path from "placed third at a hackathon" to "appeared in a recruiter's pipeline."

The shared root cause. Self-declared claims and verified evidence are treated as interchangeable. Everything above follows from that.

1.4 System Vision & Engineering Objectives

Six concrete goals, each enforced somewhere specific rather than merely stated:

#	Objective	How it is enforced
1	**No fabricated values**	Every sub-score resolves to a real number with attached evidence or an explicit *undefined*. Missing data never becomes zero
2	**Anti-gaming scoring**	Sub-scores rank against the live candidate population instead of a fixed published formula
3	**No blocking work on the API**	AI pipelines run in a separate worker; handlers release their database connection across model calls
4	**Graceful degradation**	One genuinely required dependency (the database). Queue, vector search and AI providers all have fallbacks
5	**Human review gates anything punitive**	Fraud penalties apply only to flags a person has upheld
6	**Sub-second interactive reads**	Hot-path foreign keys carry explicit indexes; the read path never invokes a model

1.5 Key Differentiators

Verified signals outrank self-declared ones, arithmetically. A skill backed by GitHub language statistics or a verified certificate earns a full point. The same skill merely typed into a resume earns 0.6. This is not a UI badge. It is the actual coefficient in the formula, which is what makes keyword-stuffing score measurably lower than substantiated experience.

Semantic matching handles near-misses. Under pure string comparison, a candidate listing "Vue.js" against a job requiring "React" scores zero overlap despite the obvious adjacency. Required skills with no literal match fall back to embedding similarity and earn partial credit, discounted because the match is inferred rather than claimed.

Job-contextual scoring. A single global talent number doesn't tell a recruiter whether someone is strong *at the thing being hired for*. The composite is recomputed per job, with coding and problem-solving sub-scores scaled by how well the candidate overlaps that specific job's requirements. The same person legitimately scores differently against a Rust systems role than a React frontend role.

Anti-gaming through population-relative scoring. A fixed formula published in the code is trivially farmable, buy stars until you clear the threshold. Sub-scores instead rank against the actual candidate population, so a star count only scores well if it beats real peers.

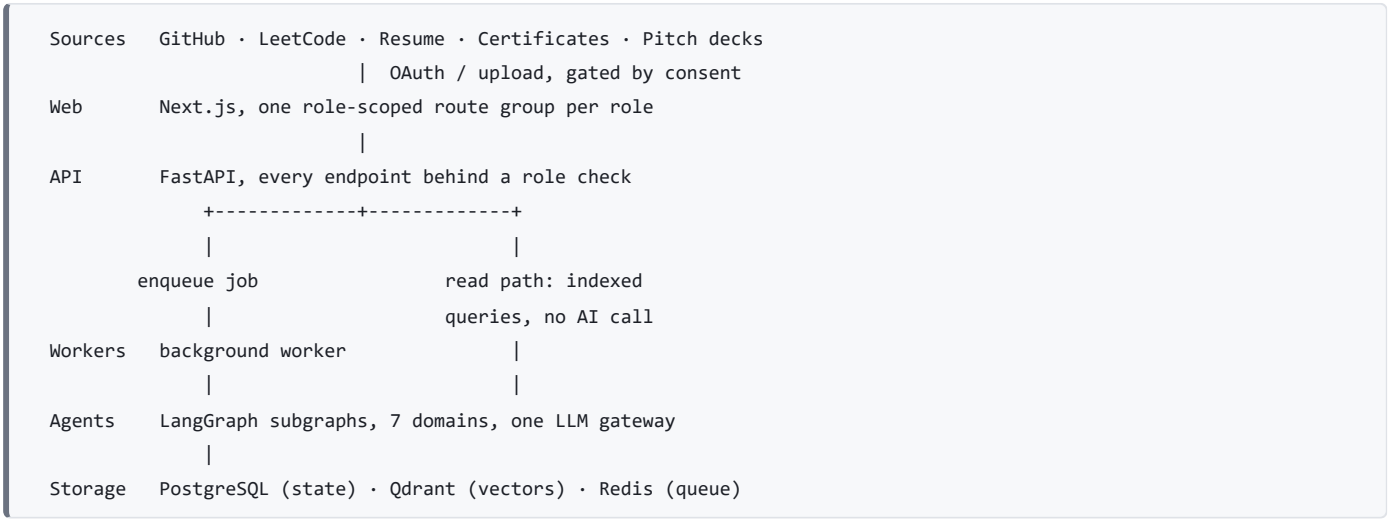
Cold start treated as a modeling problem, not an error. Missing components drop out and the remaining weights redistribute. An **Evidence Confidence** figure ships alongside every score, so a recruiter can tell a genuinely low score from a low-evidence one. The score is never withheld. It is labeled.

Explainability by design. Every match stores its component breakdown, not just the composite. The reasoning survives in the database, so a decision can be reconstructed later.

2. System Architecture & Technical Design

2.1 Architecture & Data Flow

Five layers, with one hard rule: request handlers own database access, and agent code never touches the database. Agent logic receives its data as arguments and returns values, which is what makes the scoring logic testable in isolation.



The path a candidate profile takes. An OAuth callback writes a consent record, marks the profile as processing, and queues an ingestion job. The endpoint returns immediately instead of holding the connection open for minutes. The worker then walks the pipeline: GitHub crawl, resume parse, certificate reading, profile merge, fact-check, then scoring. It records which stage it is on at each step, and the interface polls that stage to show real progress.

Long work never blocks the API. AI pipelines run in a separate worker process, and handlers release their database connection while waiting on a model. Without that, a handful of concurrent AI interviews would tie up every connection and unrelated pages would stall.

Graceful degradation. Queue unavailable? Jobs run in-process. No model provider key? Agents fall back to deterministic rules. Vector search down? Skill matching falls back to exact comparison. There is exactly one genuinely required dependency: the database. Everything else is an upgrade.

2.2 Technology Stack

Layer	Choices	Why
Backend	Python, FastAPI, SQLAlchemy	Async throughout; the AI ecosystem is Python-native
AI orchestration	LangGraph	Explicit state machines beat opaque agent loops when scoring must be auditable
Models	Multi-provider gateway (Anthropic default)	Two-tier routing: a fast model for extraction, a stronger one for judgment
Embeddings	Local sentence-transformer model	No per-call cost, no rate limit, works offline
Database	PostgreSQL	Relational integrity for evaluative data
Vector search	Qdrant	Filtered search, "semantically similar *and* remote-eligible"
Queue	Redis-backed worker	Durable background jobs with retries
Frontend	Next.js, React, TypeScript, Tailwind	Route groups map one-to-one onto roles, so the access boundary is visible in the file tree
Code sandbox	Pyodide (WebAssembly)	Candidate code runs in the browser, never on our servers
Charts	Recharts	Radar, funnel, histogram, heatmap

One choice worth calling out. Running submitted code server-side would require container isolation, syscall filtering and resource caps to be safe. Running it in the browser's WebAssembly sandbox means untrusted code never reaches our infrastructure, and a malicious submission can at worst hang the submitter's own tab.

2.3 Modular System Components

Seven agent domains, each with the same internal shape:

Domain	Responsibility
Candidate Intelligence	Resume parsing, GitHub analysis, certificate reading, profile merge, fact-check, scoring
Recruitment	Job matching, and a copilot that turns recruiter questions into structured searches
Assessment	Code verification, the live interview loop, report synthesis, contribution analysis
Pitch Analyzer	Four parallel rubric agents over pitch decks, plus cross-deck similarity
Hackathon	Links repositories to decks, computes novelty, aggregates rankings, notifies recruiters
Trust	Certificate verification, plagiarism, duplicate profiles, AI-content heuristics
Supervisor	Classifies incoming intent and routes to the right domain

2.4 Scalability Strategy

Three axes scale without architectural change. **API processes** scale horizontally, since session state lives in the token and rate-limit counters live in shared storage. **Workers** scale by running more processes, though each holds a resident embedding model, so concurrency per worker is deliberately low. **Reads** scale via database replicas for the analytics paths, which are the only genuinely read-heavy ones.

Duplicate work is prevented by deriving each job's identity from its subject, so a double-clicked "recompute" collapses into one run rather than two pipelines racing to write the same rows.

---

3. Repository Structure

A monorepo: shared code in `packages/`, runnable processes in `services/`, the web app in `apps/web`.

```
overwatch/
|-- infra/                container definitions for local services
|-- scripts/              dev helpers, seeders, index backfill
|-- packages/
|   |-- db/               data models + migrations
|   |-- shared_schemas/   typed contracts: API <-> agents <-> frontend
|   `-- prompts/          versioned prompt templates
|-- services/
|   |-- api/              request handlers, one router per domain
|   |   |-- core/         config, db, auth, rate limit, queue, AI gateway
|   |   `-- modules/      one package per feature domain
|   |-- agents/           the 7 domains above
|   |   `-- <domain>/     graph definition, nodes/, tools/, tests/
|   `-- workers/          background job runner
`-- apps/web/src/
    |-- app/              route groups: one per role + public
    |-- components/       ui, charts, feature components
    `-- lib/ hooks/       API client, sandbox runner, polling hooks
```

Every agent domain repeats the same layout: the graph defines the state machine, `nodes/` orchestrates, `tools/` holds pure functions, and `tests/` covers those functions without needing a database.

> **Reading the code:** to follow a request, start at the router. To understand a score, skip the graphs and read `tools/`, every scoring function is pure and reads top to bottom.

---

4. Feature Breakdown & User Workflows

Each feature is described as **Input → Processing → Output.**

4.1 Security & Credential Management

Authentication. Self-hosted password hashing plus signed tokens. A managed auth provider was considered and rejected, because it adds a network hop and an external dependency to every authenticated request. That is a poor trade for five roles and a token.

- **Input:** email, password, role at signup; email and password at login.
- **Processing:** passwords are hashed with a per-password salt. Verification is wrapped so that a malformed stored hash reads as a failed login, never as a server error that leaks the malformation.
- **Output:** a signed token carrying the subject and expiry.

Authorization. Five roles, constrained at the database *and* checked at the API. Authorization is declared in each route's signature rather than performed in its body, so an endpoint cannot accidentally skip it through an early return:

```
@router.get("/admin/fraud-review-queue")
async def fraud_review_queue(user = Depends(require_role(Role.admin))):
    ...
```

Role membership alone is never sufficient. Ownership is a separate check. A recruiter with a valid token still cannot read a job posted by a different recruiter.

Consent. Anything privacy-sensitive requires a typed consent record first, covering resume parsing, interviews, photo hashing and assessments. Records store when consent was granted, from where, and under which terms version. Revocation writes a timestamp rather than deleting the row, so the audit trail survives.

Rate limiting. Shared counters keyed on the authenticated user, falling back to client IP. Counters are shared rather than per-process, because a per-process limiter running four API workers silently multiplies the effective limit by four.

4.2 Candidate Intelligence

A candidate connects GitHub, uploads a resume and certificates. The pipeline crawls repositories, parses the resume, reads certificates, and merges everything into one profile.

The merge step is where contradictions surface, a resume claiming five years of Go against a GitHub account with no Go repositories. Conflicts are shown to the candidate rather than silently resolved in favor of one source.

Output: nine sub-scores, a composite, an evidence receipt showing which artifacts produced which score, and skill badges backed by named sources.

4.3 Job Matching

A recruiter posts a job. The engine embeds the description, retrieves a candidate pool through combined vector and filter search, scores each pair, and generates a plain-language explanation of the fit.

Recruiters see the component breakdown, skill overlap, semantic similarity, experience, talent alignment, never a bare percentage.

4.4 Skill Verification

Candidates solve problems in an in-browser editor with instant feedback against visible test cases. On submission, static analysis plus a model review scores correctness, efficiency and style. Timed, unseen problems are the hardest signal to fake, which is why assessments carry the heaviest weight in the coding sub-score.

4.5 AI Interview

A live turn-based interview: the agent asks, the candidate answers by voice, the agent evaluates the answer and decides whether to follow up or move on. Output is a report with per-competency scores and direct evidence quotes tied to each one. Requires explicit consent, recorded before the session starts.

4.6 Hackathon-to-Hiring

Organizers create an event and import teams. Teams submit a repository and a pitch deck. The deck is analyzed by four parallel rubric agents covering problem clarity, innovation, technical feasibility and presentation. Judges score against a rubric, and a composite ranking combines judge scores, pitch quality, repository quality and cross-event novelty.

Organizers can reweight those components per event, different events value judging and novelty differently, and one hardcoded distribution would make the feature unusable for half of them.

The hiring bridge: top performers surface automatically in the feeds of recruiters watching that event.

4.7 Trust & Fraud Layer

Four independent checks: certificate verification against a trusted-issuer registry, code plagiarism detection, duplicate-profile matching, and AI-generated-content heuristics.

> **Important:** Every check is advisory. A flag affects a candidate's authenticity score **only after a human reviewer upholds it.** Detection alone carries no penalty.

The AI-content check is deliberately the weakest-weighted signal in the platform, shipped with a capped confidence level and an explicit statement that false positives are expected, particularly for non-native English writers. Presenting a statistical heuristic as proof of misconduct would be the single most damaging thing this platform could do to a candidate.

4.8 Role-Based Interfaces

Five roles, each with its own scoped section of the app, so the access boundary is visible in the structure rather than only enforced at runtime.

Who uses it	Features
Candidate	Talent dashboard · evidence receipts · GitHub activity heatmap · skill badges · career guidance and skill gaps · learning roadmap · salary range · resume builder · coding assessments · AI interview · public portfolio
Recruiter	Plain-language search · job posting · match lists with score breakdowns · drag-and-drop hiring pipeline · top-performer feed from hackathons · hiring analytics
Organizer	Create events · import teams · set judge and rubric weights · track submissions · finalize leaderboards
Judge	Scoring queue · rubric scoring · PPT analysis prepared in advance · repository summary
Admin	User and role management · audit log · fraud review queue · trusted issuer registry

Who uses it	Features
Public	Candidate portfolios · hackathon leaderboards, no login needed

Public portfolios are opt-in and default to unpublished, so no candidate is exposed by inaction.

Long-running work reports real progress through the recorded pipeline stage rather than an indeterminate spinner. A four-minute crawl of a large GitHub account is a normal outcome, not a fault, but a spinner that never changes is indistinguishable from a hang.

---

5. Quantitative Framework

5.1 Core Formulations

The central problem is **scoring under incomplete information.** Almost every candidate is missing something, no assessments yet, thin commit history, no hackathon record. Treating absence as zero builds a system that punishes newcomers and rewards nothing but tenure.

The renormalization rule

When a signal is missing, its weight is redistributed across the signals that remain:

$$S = \frac{\sum_{i \in A} w_i S_i}{\sum_{i \in A} w_i}$$

where A is the set of signals actually available. The weights always sum to 1, no matter how many dropped out.

If *nothing* is available, the result is **undefined, not zero.** That distinction is the single most important design constraint in the scoring layer. Returning zero would be arithmetically convenient and semantically catastrophic: a candidate with no data would look identical to one who genuinely scored zero.

```
function composite(terms):
    available = [(weight, value) for each term where value exists]
    if available is empty:
        return UNDEFINED          # never 0
    return weighted_average(available)
```

The Talent Score

Nine sub-scores feed one composite:

Sub-score	Weight	Primary source
Coding Ability	0.16	Commits, code quality, assessments
Problem Solving	0.16	Assessment results
Project Quality	0.12	Repository analysis
Innovation	0.12	Project novelty
Open Source Contributions	0.10	Merged PRs to external repos
Hackathon Performance	0.10	Platform and self-reported results
Technical Consistency	0.08	Commit cadence over time
Community Participation	0.08	Stars, external contributions
Leadership	0.08	Owned repositories, PR reviews

Five of the nine are computed by deterministic rules with no model call at all. AI is reserved for judgments that genuinely require reading code or prose; anything reducible to arithmetic stays arithmetic, cheaper, faster, reproducible.

Evidence Confidence ships alongside as `available signals ÷ 9`. A candidate scoring 72 on three signals and one scoring 72 on all nine are very different propositions, and a recruiter should be able to tell them apart.

The Match Score

$$M = 0.35\,(\text{skill overlap}) + 0.30\,(\text{semantic similarity}) + 0.15\,(\text{experience fit}) + 0.20\,(\text{talent alignment})$$

Skill overlap is where the anti-gaming intent is most visible:

```
for each required skill:
    if candidate has it, verified        -> credit 1.0
    else if candidate claims it          -> credit 0.6
    else if a similar skill is close     -> credit 0.6 x similarity
    else                                 -> credit 0
overlap = 100 x total credit / number of required skills
```

Experience fit saturates at the stated minimum, a twenty-year veteran is not a "200% match" for a two-year role, so exceeding the bar earns full credit and stops there.

> **On the weights:** these are reasoned starting points, not empirically validated constants. Validating them properly needs outcome data on who advanced, who was hired and who succeeded. That is exactly what the platform accumulates over time. The schema stores every component separately so that turning this into a supervised learning problem later requires no migration.

5.2 Statistical Methods

The statistical work is distributional rather than stochastic, ranking against a population, weighting by recency, trimming outliers.

Percentile normalization. A raw count becomes a rank against the actual candidate pool:

$$\mathrm{pct}(x) = \frac{100}{|\mathcal{P}|}\,\big|\{v \in \mathcal{P} : v \le x\}\big|$$

where $\mathcal{P}$ is the population of observed values. Below a 30-candidate floor the ranks swing wildly on each new profile, so a fixed fallback formula applies instead. The code does not claim a precision it does not have.

Recency decay. Signals halve in weight every six months:

$$w(t) = e^{-\lambda t}, \qquad \lambda = \frac{\ln 2}{180}$$

with t in days, floored at zero. Someone prolific two years ago and silent since should not score identically to someone shipping now.

Winsorization. Assessment values are clipped to their own 5th and 95th percentiles before ranking, so one candidate acing a trivially easy assessment doesn't skew the distribution against everyone else.

Consistency over volume. Technical consistency inverts the coefficient of variation of recency-weighted weekly commit counts, so a steady cadence scores higher than the same total delivered in one burst. Fewer than four active weeks returns *undefined*, because that is a cold start rather than a poor result.

5.3 Formulation Summary Matrix

Metric	Formula	Range	Undefined when
Renormalized mean	$\sum_{i\in A} w_i S_i / \sum_{i\in A} w_i$	0–100	No signals available
Talent Score	Weighted sum of 9 sub-scores	0–100	All sub-scores absent
Evidence Confidence	$\lvert A\rvert/9$	0–1	Never
Match Score	$0.35O + 0.30\Sigma + 0.15E + 0.20T$	0–100	Never
Skill Overlap	Credit-weighted required-skill ratio	0–100	No required skills
Experience Fit	$100\min(y_C/y_{\min},\,1)$	0–100	No stated minimum
Percentile Rank	$100\lvert\{v \le x\}\rvert/\lvert\mathcal{P}\rvert$	0–100	Population under 30
Recency Weight	$e^{-t\ln 2/180}$	0–1	Never
Technical Consistency	Inverted CV, with staleness penalty	0–100	Under 4 active weeks
Authenticity	$100 - \sum(\text{upheld flag penalties})$	0–100	Never

Variables: A = set of available signals · w_i = configured weight · S_i = component score · $\mathcal{P}$ = population of observed values · y_C = candidate years, $y_{\min}$ = job minimum · t = days elapsed · O, Σ, E, T = overlap, semantic similarity, experience, talent alignment.

---

6. Data Model & Security

6.1 Schema Overview

Relational state in PostgreSQL, organized into seven domains:

Domain	Holds
Identity	Users, organizations, roles, consent records, audit log
Candidate	Profiles, GitHub snapshots, certifications, talent scores, badges
Recruitment	Jobs, applications with pipeline stages, match scores
Assessment	Problem definitions, submissions, interview sessions and turns
Hackathon	Events, teams, submissions, judge scores, rankings
Trust	Fraud flags, authenticity scores, disputes, trusted issuers
Operations	Agent run history, event outbox

Three conventions run throughout:

- **Every score stores its components,** not just the composite. That is what makes an explanation reconstructible months later.
- **Consent is a record, not a boolean.** Revocation writes a timestamp rather than deleting the row, so the audit trail survives. A candidate can revoke and re-grant, so lookups take the most recent record.
- **Semi-structured evidence lives in JSON columns** (skills, conflicts, score breakdowns); anything queried or joined stays relational.

Two schema details carry real weight. Score columns are **nullable on purpose**, because that is what separates "no evidence" from "scored zero." And foreign keys get **explicit indexes**, since PostgreSQL indexes primary keys automatically but not foreign keys, and the recruiter candidate pool reads across those on every request.

Enum-like values use CHECK constraints rather than database enum types, because adding a value to a CHECK is a one-line migration.

6.2 Access Control

Four layers, application first:

Layer	Mechanism
Role	Every endpoint declares its required role in its signature, so it cannot be skipped by an early return
Ownership	Checked separately from role. A recruiter with a valid token still cannot read another recruiter's pipeline
Identity	Candidate endpoints read the profile from the token and never accept an ID from the client, which removes a whole class of access bug instead of patching it per endpoint
Database	Row-level security acts as a second net, so a query that escapes the application still cannot cross tenants

Candidates can never write their own scores; that path is limited to the service context. Audit logs have no update or delete policy at all, so both are denied by default, which is the right property for an audit trail.

Passwords are salted hashes. Error reporting excludes personal data. Every admin action is logged with actor, action and target. Public portfolios are opt-in and default to unpublished.

---

7. Development Methodology & Setup

7.1 Build Phases

The build order follows one principle: **the parts that are hardest to correct later come first.**

Phase	Focus	Why in this order
1. Foundation	Data model, authentication, role gating	Nothing can be tested behind a role boundary until the boundary exists
2. Scoring core	The pure scoring functions, with tests	This is the math that decides people's job prospects. It has no external dependencies, so it can be fully tested before any real data exists

Phase	Focus	Why in this order
3. Agent pipelines	The AI workflows, one domain at a time	Candidate intelligence first, since the Talent Score feeds almost everything downstream
4. Interfaces	The five role-scoped surfaces	Built against real endpoints rather than mock data, which has a way of surviving into production
5. Hardening	Queue durability, indexes, rate limits, degradation paths	Given its own budget rather than whatever time remains, that is what stops it being cut

Scoring logic is kept separate from everything else. The functions that compute a score take their data as plain arguments and return plain values, no database, no network. That makes them testable in isolation and reviewable by someone who doesn't know the rest of the codebase. Testing effort concentrates here rather than spreading thin.

Validation goes beyond unit tests. Score functions are checked at their boundaries (a percentile rank of the population maximum must be exactly 100). Cold-start behavior is checked for every possible subset of missing signals. Sub-scores are cross-checked by hand against real GitHub profiles with known characteristics, the only method that catches a formula that is internally consistent but ranks people wrongly. And each external dependency is removed in turn to confirm the system returns a clear error rather than a fabricated number.

> **The rule behind all of it:** a crash announces itself; a plausible wrong score does not. The dangerous failures are the ones that produce believable output from bad input, so the architecture is built to make those structurally difficult, undefined instead of zero, typed errors instead of fallback values, and human review gating anything that penalizes a candidate.

7.2 Prerequisites & Installation

Requirement	Minimum	Verify with
Python	3.12	`python --version`
Node.js	20 (22 LTS preferred)	`node --version`
Docker + Compose	v2	`docker compose version`
RAM	8 GB (16 GB comfortable)	Task manager

The memory figure is real, not padding. The embedding model is roughly 1.3 GB resident and loads in both the API and each worker.

Setup, in order:

```
git clone <repository-url> overwatch && cd overwatch
docker compose -f infra/docker-compose.yml up -d    # database, vector store, queue
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cp .env.example .env                                # then set JWT_SECRET_KEY
python -m alembic upgrade head
python scripts/seed_db.py && python scripts/seed_candidates.py
python scripts/ensure_qdrant_indexes.py             # idempotent
cd apps/web && npm install
```

Then run four processes:

```
uvicorn services.api.main:app --reload --port 8000   # API
python -m services.workers.runner                    # background worker
cd apps/web && npm run dev                           # web on :3000
```

> **Tip:** run the second seed script. Percentile ranking needs a 30-candidate population before it engages, and below that the scoring quietly takes a different path than it would in production.

7.3 Key Configuration

Local defaults ship ready to run, so a first start needs almost nothing set.

Variable	Required	Default	Notes
`DATABASE_URL`	Yes	local container	The one hard dependency
`JWT_SECRET_KEY`	**Yes**	*none*	Generate with `secrets.token_urlsafe(48)`
`ANTHROPIC_API_KEY`	No	empty	Absent: agents fall back to fixed rules
`REDIS_URL`	No	local container	Absent: jobs run in-process
`QDRANT_URL`	No	local container	Absent: exact skill matching only
`GITHUB_CLIENT_ID` / `_SECRET`	No	empty	Only for GitHub sign-in
`RATE_LIMIT_PER_MINUTE`	No	60	Per user or IP

> **Important:** `JWT_SECRET_KEY` deliberately has no default. The application refuses to start without it, because a framework-supplied default signing key is one of the most common ways applications ship insecure.

8. Verification, Testing & Guardrails

Automated checks.

```
npx tsc --noEmit                          # TypeScript, strict mode
npm run lint && npm run build             # frontend
ruff check services packages              # backend lint
python -m alembic upgrade head --sql      # migration chain validity, without executing
pytest services/agents -v                 # scoring function tests
```

The type check earns its place because the API contract spans two languages: rename a backend field without updating the frontend type, and the compiler catches it at build time instead of leaving an `undefined` to surface in production.

Failure isolation. Every dependency has a defined behaviour when it goes away:

Dependency	If it fails	Effect
Database	Request errors	Total. The one hard dependency
Queue	Falls back to in-process	Works, but loses durability and retries
Vector search	Typed error raised	Semantic search unavailable; exact matching still works
AI provider	Typed error, HTTP 503	AI features unavailable; rule-based scores unaffected
File storage	Typed error, HTTP 503	Uploads unavailable; everything else fine

The pattern is uniform: a named error for the missing dependency, surfaced as a specific status code. Never a generic 500, and never an invented value standing in for a real one.

Scoring guardrails. Missing signals renormalize rather than zero-fill. Every score is clamped to its stated range. Percentile ranks fall back below a 30-candidate population. Fraud flags need human review before affecting anything. Every score keeps its component breakdown.

Input handling. Uploads are size-capped before being streamed anywhere. Candidate code runs only in the browser sandbox. Interview sessions carry a token budget so a long transcript cannot grow the context window without bound. All request bodies are schema-validated, so malformed input returns a field-level 422 rather than reaching handler code.

9. Deployment

Web. The app uses server components, so it needs a Node runtime rather than a static host. Security headers are set at the edge, including cross-origin isolation on WebAssembly responses.

> **Important:** the code sandbox needs cross-origin isolation headers to work. Without them it fails in production while continuing to work locally, because localhost is treated as a secure context. This is the kind of thing that surfaces during a demo.

API and worker. One container image, two entrypoints, running as a non-root user behind a reverse proxy that terminates TLS. Building them separately would double build time and invite version skew between two processes that have to agree on the database schema.

The container health check needs a 90-second grace period, because the embedding model loads at startup. A shorter one marks the container unhealthy during a normal boot and sends the orchestrator into a restart loop.

A migration step runs once and exits, and both API and worker wait for it to finish, so neither ever starts against an un-migrated schema. Database, vector store and queue sit in the same region as the API. Putting them elsewhere makes every query pay a public-internet round trip, which reads as slow pages while query timings look perfectly fine.

Mobile. A wrapper around the same web build rather than a reimplementation. Tokens go to the platform keystore, not local storage. Two features stay web-only: the code sandbox needs browser features the mobile webview does not reliably support, and server-rendered portfolios need a server runtime. Both are shown as "open on web" rather than shipped broken.

---

10. Roadmap & License

10.1 Roadmap

Near-term

- **Validate the scoring weights.** Every weight is currently a reasoned default. With outcome data, who advanced, who was hired, this becomes a supervised learning problem rather than a judgment call.
- **Score versioning.** Recompute historical scores under a new formula while preserving the old ones, so a candidate's trajectory doesn't jump discontinuously when weights change.
- **Replace the AI-content heuristic** with a proper perplexity model. The current statistic is weak and labeled as such.

Medium-term

Recruiter-defined scoring profiles, so a startup weighting scrappiness and an enterprise weighting consistency can share one platform. Bias auditing across demographic proxies. Integrations with existing ATS tools, since no recruiter is abandoning theirs. More assessment languages beyond Python.

Longer-term

Talent trajectory modeling over time, team composition analysis for organizers, and multi-language resume parsing.

Deliberately not on this roadmap: automated rejection. The platform ranks, explains and surfaces evidence, a human decides. Every guardrail described here assumes a person in the loop: human review gating fraud penalties, confidence labels on scores, component breakdowns instead of bare numbers. Automating the decision away would invalidate the design.

10.2 License

Dependencies are permissively licensed (MIT, BSD, Apache-2.0), with one exception: the PDF extraction library is AGPL-3.0. That is fine for an open MIT distribution, but a closed-source commercial deployment would need either a commercial license or a swap to a permissive alternative. The dependency is isolated to resume text extraction, so that swap stays contained.

Data and attribution. The salary model trains on the public Stack Overflow Developer Survey. GitHub data is read only under an explicit OAuth grant, within approved scopes. Any processing of personal data needs a recorded, revocable consent record. Generated documents belong to the candidate who asked for them.
